

Ingeniería de soluciones con AI

Semestre 2026-1

Docente:
Francisco Macaya

Noticias: LangChain Lanza LangSmith Engine.



Nota: <https://www.langchain.com/blog/introducing-langsmith-engine>

LangSmith Engine: Los fallos terminan siendo oportunidades.

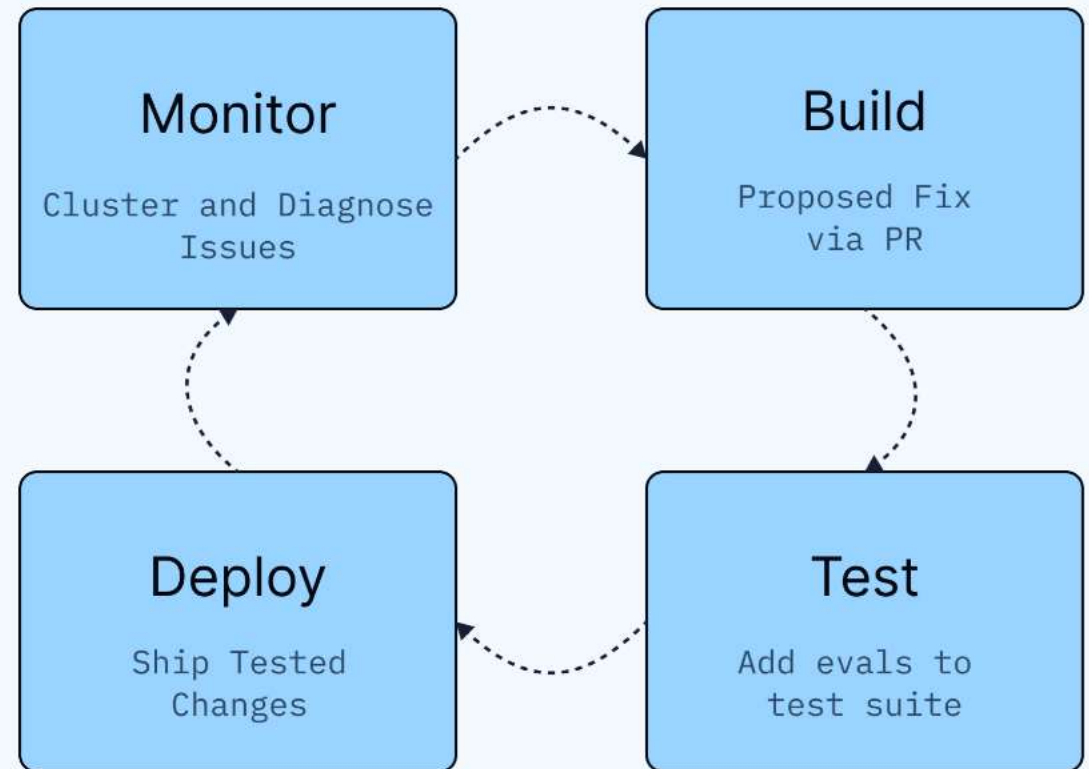
Engine surfaces this as a single named issue, "Agent fails to handle subscription cancellation requests accurately." It shows you the severity (high, affecting 12% of support sessions this week), the timeline (started four days ago, correlating with a recent deployment), and links to the specific traces as evidence.

With your repository connected, Engine reads the relevant code and identifies the root cause. The cancellation tool description is ambiguous, causing the agent to attempt cancellation when users are only asking about their options. Engine drafts a PR with a targeted fix to the tool description.

To keep tracking this behavior going forward, Engine proposes a custom online evaluator scoped to this exact issue, so if the failure pattern recurs after the fix ships, the issue gets resurfaced automatically with updated details.

Engine also pulls the failing traces into a dataset for your offline eval suite, with per-example criteria that define what the correct output should contain. The failures that made it to production become the test cases that keep them out.

That's the full cycle, run autonomously and surfaced for your review. Production signal becomes a clustered issue, then a diagnosed root cause, a proposed fix, and eval coverage.



¿Que es la Observabilidad?

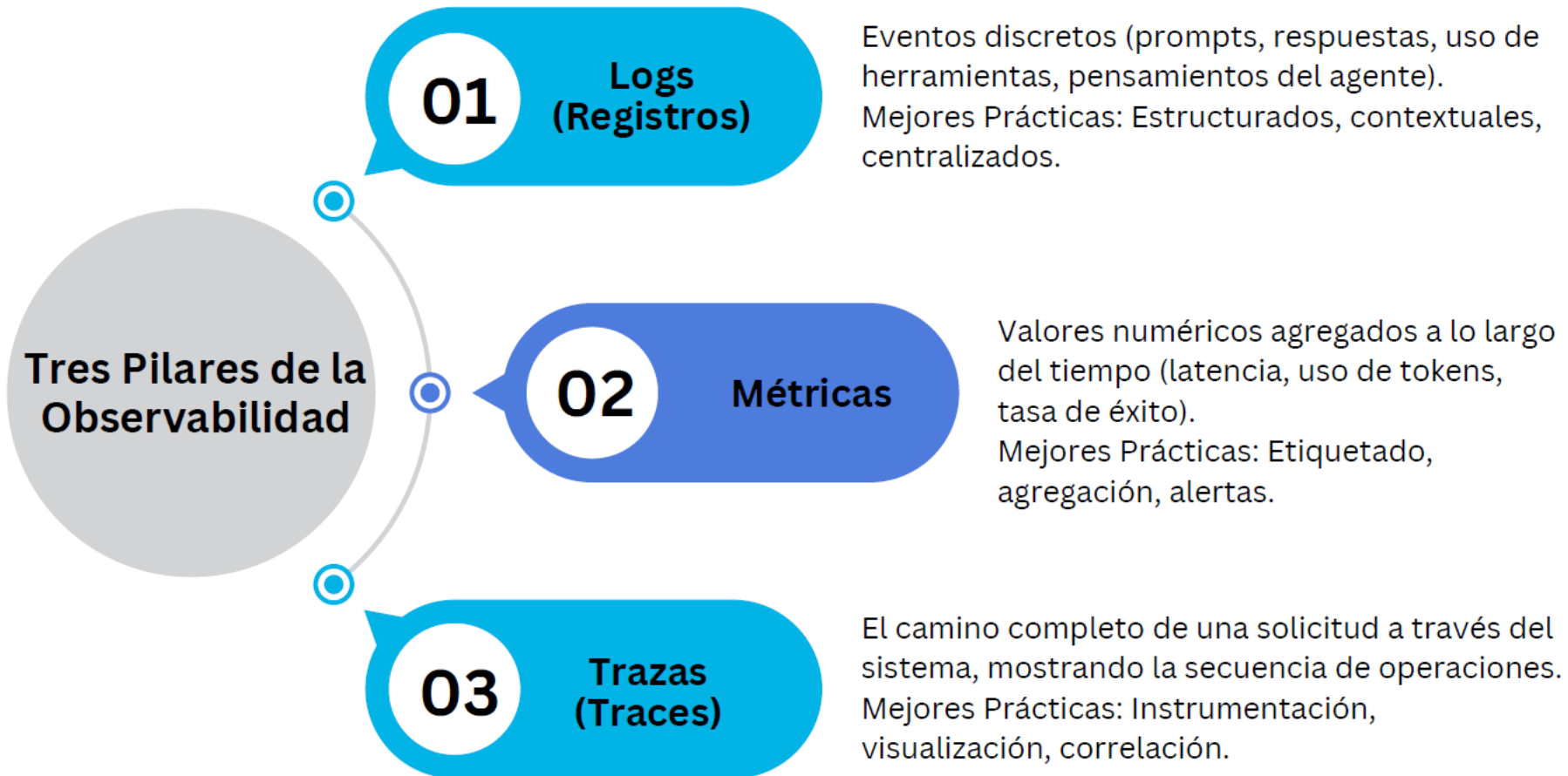
Observability is the ability to understand the internal state of a system by examining its outputs.

In software, this is typically achieved by analyzing telemetry data such as traces, metrics, and logs.

To make a system observable, it must be instrumented. That is, the code must emit traces, metrics, or logs. The instrumented data must then be sent to an observability backend.



Tres “Stones”: Logs, Métricas e Traces(Trazas).



Métricas: “Click” a métricas de performances.

-**Latency (End-to-End Latency):** Tiempo total desde la solicitud hasta la respuesta completa. Es más importante para respuestas cortas y no transmitidas en tiempo real (non-streamed).

-**Requests per Second (RPS):** cuántos usuarios concurrentes puede manejar el sistema.

-**Costo por interacción** = (Prompt tokens × precio) + (Completion tokens × precio)

- Prompt tokens** — todo lo que entra al modelo: system prompt, historial de conversación y mensaje del usuario

- Completion tokens** — los tokens generados por el modelo como respuesta

Ejercicios: Mide, evalúa y plantea mejoras.

1- Utilizar la API descrita en el README para evaluar costos, modelos y latencia de una solución.

2- En base a lo encontrado, ¿Qué mejoras se puede hacer en el código para poder disminuir la latencia de la solución?